

HDOC 32

Name: Keshav Mittal

SAP ID: 590032934

Batch: 16

1. Algorithm

1. Start main program.
2. Display the menu with options 1 to 3.
3. Read the user's choice and use a switch statement.
4. Case 1 (Addition Commutativity):
 - Read dimensions r and c . Declare four VLAs: $A[r][c]$, $B[r][c]$, $AB[r][c]$, $BA[r][c]$.
 - Read elements for matrices A and B .
 - Loop through elements to compute $AB[i][j] = A[i][j] + B[i][j]$ and $BA[i][j] = B[i][j] + A[i][j]$.
 - Print both matrices. If they match completely, output that matrix addition is commutative.
5. Case 2 (Multiplication Non-Commutativity):
 - Read dimension n for square matrices (to ensure both $A*B$ and $B*A$ are mathematically valid for comparison).
 - Declare four VLAs: $A[n][n]$, $B[n][n]$, $AB[n][n]$, $BA[n][n]$.
 - Read elements for matrices A and B . Initialize AB and BA elements to 0.
 - Use triple nested loops to compute $AB[i][j] += A[i][k] * B[k][j]$ and $BA[i][j] += B[i][k] * A[k][j]$.
 - Print both matrices. If they do not match, output that matrix multiplication is not commutative.
6. Case 3 (Exit): Terminate the program.
7. Stop

2. Test-Case Table

TC ID	Scenario	Inputs	Expected Output
TC-01	Addition Commutativity	Choice: 1, 2x2. $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$A+B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$, $B+A = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$. Addition is commutative.
TC-02	Multiplication Non-Commutativity	Choice: 2, 2x2. $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$	$A*B = \begin{bmatrix} 0 & 1 \\ 0 & 3 \end{bmatrix}$, $B*A = \begin{bmatrix} 3 & 4 \\ 0 & 0 \end{bmatrix}$. Multiplication is not commutative.
TC-03	Exit	Choice: 3	Exiting program...

3. C Program Source Code

```
#include <stdio.h>

void proveAdditionCommutative(void);
void proveMultiplicationNotCommutative(void); void readMatrix(int
r, int c, int mat[r][c]); void printMatrix(int r, int c, int mat[r][c]);
int compareMatrices(int r, int c, int m1[r][c], int m2[r][c]);

int main() {
    int choice; do {
        printf("\n--- Matrix Commutative Properties Menu ---\n"); printf("1. Prove matrix addition is
commutative\n"); printf("2. Prove matrix multiplication is not commutative\n"); printf("3.
Exit\n");
        printf("Enter your choice: "); scanf("%d", &choice);

        switch (choice) { case 1:
            proveAdditionCommutative(); break;
            case 2:
            proveMultiplicationNotCommutative(); break;
            case 3:
            printf("Exiting program...\n"); break;
            default:
            printf("Error: Invalid choice.\n");
        }
    } while (choice != 3);

    return 0;
}

void readMatrix(int r, int c, int mat[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { scanf("%d", &mat[i][j]);
    }
}
}

void printMatrix(int r, int c, int mat[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { printf("%d\t", mat[i][j]);
    }
    printf("\n");
}
}

int compareMatrices(int r, int c, int m1[r][c], int m2[r][c]) {
```

```

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) { if (m1[i][j] !=
            m2[i][j]) {
                return 0;
            }
        }
    }
    return 1;
}

void proveAdditionCommutative(void) { int r, c;
    printf("Enter rows and columns for matrices A and B: "); scanf("%d %d", &r, &c);
    if (r <= 0 || c <= 0) return;

    int A[r][c], B[r][c], AB[r][c], BA[r][c];

    printf("Enter elements of Matrix A:\n"); readMatrix(r, c, A);
    printf("Enter elements of Matrix B:\n"); readMatrix(r, c, B);

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) { AB[i][j] = A[i][j] + B[i][j];
            BA[i][j] = B[i][j] + A[i][j];
        }
    }

    printf("Matrix A + B:\n"); printMatrix(r, c, AB);

    printf("Matrix B + A:\n"); printMatrix(r, c, BA);

    if (compareMatrices(r, c, AB, BA)) {
        printf("Since A + B == B + A, matrix addition is commutative.\n");
    } else {
        printf("Error in logic.\n");
    }
}

```

```

void proveMultiplicationNotCommutative(void) { int n;
    printf("Enter the size 'n' for square matrices A and B (n x n): "); scanf("%d", &n);
    if (n <= 0) return;

    int A[n][n], B[n][n], AB[n][n], BA[n][n];

    printf("Enter elements of Matrix A:\n"); readMatrix(n, n, A);
    printf("Enter elements of Matrix B:\n"); readMatrix(n, n, B);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {

```

```

        AB[i][j] = 0;
        BA[i][j] = 0;
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
            AB[i][j] += A[i][k] * B[k][j];
            BA[i][j] += B[i][k] * A[k][j];
        }
    }
}

printf("Matrix A * B:\n");
printMatrix(n, n, AB);

printf("Matrix B * A:\n");
printMatrix(n, n, BA);

if (!compareMatrices(n, n, AB, BA)) {
    printf("Since A * B != B * A, matrix multiplication is not commutative.\n");
} else {
    printf("For these specific matrices, A * B == B * A. Try different values to
prove non-commutativity.\n");
}
}

```

4. Compilation and Execution

```

(kali@kali)-[~/liak]
$ cc hdoc_32.c

(kali@kali)-[~/liak]
$ ./a.out

— Matrix Commutative Properties Menu —
1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit
Enter your choice: 1
Enter rows and columns for matrices A and B: 2 3 3 2
Enter elements of Matrix A:
2 3 4 5
Enter elements of Matrix B:
5 4 3 2

```